

Università degli Studi di Torino
Corso di Laurea Magistrale in Informatica

Tecnologie del Linguaggio Naturale

Parte 3 – Prof. Luigi Di Caro

Lab 5

Hybrid Search RAG

Retrieval-Augmented Generation con ricerca ibrida semantica e lessicale

Gruppo di lavoro:

Studente	Matricola
Alessandro Olivero	915069
Samuele Iudice	1235245
Andrea Franco	1226739

Docente: Prof. Luigi Di Caro

Anno Accademico: 2025/2026

Indice

1 Introduzione

Questo laboratorio affronta il problema della **Retrieval-Augmented Generation (RAG)**, un paradigma che potenzia i modelli di linguaggio combinando un meccanismo di *retrieval* su una knowledge base esterna con la capacità generativa di un Large Language Model (LLM). L'obiettivo è confrontare due strategie di retrieval su un corpus scientifico NLP:

- **RAG base**: recupero puramente semantico tramite embeddings SciBERT e indice FAISS.
- **RAG ibrido**: combinazione del retrieval semantico con BM25 lessicale, fusi tramite Reciprocal Rank Fusion (RRF).

L'esperimento principale varia la dimensione del corpus da 1 000 a 44 949 documenti, misurando l'impatto sulle prestazioni con quattro metriche ispirate al framework RAGAS: Precision@5, Context Relevance, Answer Relevance e Faithfulness. Si analizza inoltre la **robustezza** del sistema a parafrasature delle query.

Il dataset usato è **MaartenGr/arxiv_nlp**, una collezione di paper scientifici NLP da arXiv, disponibile su Hugging Face. Il modello di linguaggio è **Phi-3.5-mini-instruct** di Microsoft, eseguito interamente in locale su CPU.

2 Background Teorico

2.1 Retrieval-Augmented Generation (RAG)

Un sistema RAG si articola in tre componenti:

1. **Retriever**: dato una query, recupera i documenti più rilevanti da una knowledge base.
2. **Context assembly**: i documenti recuperati vengono concatenati come contesto.
3. **Generator**: un LLM produce una risposta condizionata dal contesto e dalla query.

Il vantaggio principale rispetto a un LLM stand-alone è la capacità di accedere a informazioni aggiornate e specifiche di dominio senza ri-addestrare il modello, riducendo al contempo le allucinazioni.

2.2 Retrieval semantico con FAISS e SciBERT

Il retrieval semantico si basa su **dense embeddings**: ogni documento e ogni query vengono proiettati in uno spazio vettoriale denso, e il retrieval è una ricerca dei vicini più prossimi (Nearest Neighbor Search).

Si usa **SciBERT** (`allenai/scibert_scivocab_uncased`) come modello di embedding: un BERT pre-addestrato su 1,14 milioni di paper scientifici da Semantic Scholar. Produce vettori di dimensione 768, normalizzati a norma unitaria per l'uso con similarità coseno.

Nota sulla scelta del modello: SciBERT non è un modello **sentence-transformers** nativo, cioè non è stato fine-tuned con un obiettivo di similarità tra frasi/documenti; caricandolo con `SentenceTransformer(...)` si ottiene un mean pooling di default (con relativo

warning a runtime), che funziona ma può soffrire del problema noto di *anisotropia* degli embedding BERT non specializzati per il retrieval. La scelta resta difendibile per il vocabolario specializzato nel dominio scientifico, ma un'alternativa più indicata sarebbe `allenai/specter` (o `specter2`), una variante di SciBERT già addestrata esplicitamente per la similarità tra paper scientifici (tramite citazioni come segnale di supervisione).

L'indice di ricerca è costruito con **FAISS** (Facebook AI Similarity Search), che implementa **IndexFlatIP** (inner product su vettori normalizzati, equivalente alla similarità coseno) e consente ricerca efficiente su decine di migliaia di vettori.

2.3 Retrieval lessicale con BM25

BM25 (Best Match 25) è un modello di recupero probabilistico basato sulla frequenza dei termini. Per ogni query q e documento d , il punteggio è:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

dove $f(t, d)$ è la frequenza del termine t nel documento d , $|d|$ è la lunghezza del documento, avgdl è la lunghezza media del corpus, e $k_1 = 1.5$, $b = 0.75$ sono iperparametri standard.

A differenza del retrieval semantico, BM25 è efficace per query con termini tecnici specifici (es. acronimi, nomi propri) che gli embeddings tendono a generalizzare.

2.4 Reciprocal Rank Fusion (RRF)

La **Reciprocal Rank Fusion** è una tecnica di fusione dei risultati di più retriever. Dati i ranking prodotti da n retriever, il punteggio RRF di un documento d è:

$$\text{RRF}(d) = \sum_{r=1}^n \frac{1}{k + \text{rank}_r(d)}$$

con $k = 60$ (costante di smorzamento). I documenti vengono ri-ordinati per punteggio RRF decrescente. Questo approccio è robusto perché non richiede normalizzazione degli score eterogenei e valorizza i documenti consistentemente rilevanti per più retriever.

2.5 Phi-3.5-mini-instruct

Il modello generativo è **Phi-3.5-mini-instruct** di Microsoft: un LLM da 3.8B parametri ottimizzato per il ragionamento e l'instruction-following. Viene eseguito in modalità *float32* su CPU, con generazione limitata a 300 token. Il template di chat segue il formato Phi-3 con ruoli `system` e `user`.

2.6 Metriche di valutazione ispirate a RAGAS

Si implementano quattro metriche per valutare il sistema RAG, ispirate al framework RAGAS (Retrieval-Augmented Generation Assessment System):

- **Precision@5**: percentuale di documenti rilevanti (per gold standard) tra i top-5 recuperati.

$$P@5 = \frac{|\{d \in R_5 : d \text{ rilevante}\}|}{5}$$

- **Context Relevance (CR)**: similarità coseno media tra l'embedding SciBERT della query e gli snippet dei documenti recuperati. Misura quanto il contesto è semanticamente pertinente alla domanda.

$$CR = \frac{1}{|R|} \sum_{d \in R} \cos(\text{enc}(q), \text{enc}(d[:200]))$$

- **Answer Relevance (AR)**: similarità coseno tra l'embedding della query e l'embedding della risposta generata dall'LLM. Misura se la risposta è centrata sulla domanda.

$$AR = \cos(\text{enc}(q), \text{enc}(a))$$

- **Faithfulness proxy (FF)**: percentuale di parole contenutistiche della risposta (escluse le stopwords) che compaiono nel testo dei documenti recuperati. Misura in modo approssimato quanto la risposta è fondata sul contesto.

$$FF = \frac{|\{w \in a_{\text{content}} : w \in C\}|}{|a_{\text{content}}|}$$

dove C è il testo concatenato dei documenti di contesto.

- **Robustezza**: per un sottoinsieme di query, si calcolano le parafrasature e si misura l'overlap@5 tra i risultati della query originale e quelli delle parafrasature:

$$Rob = \frac{1}{|P|} \sum_{p \in P} \frac{|R_5(q) \cap R_5(p)|}{5}$$

Un valore alto indica che il retrieval è stabile a variazioni di formulazione della query.

3 Architettura del Sistema

Il sistema si struttura in tre livelli:

1. **Livello dati**: caricamento del dataset arXiv-NLP, campionamento di N documenti, costruzione del testo come concatenazione di titolo e abstract.
2. **Livello retrieval**: indice FAISS (semantico) + indice BM25 (lessicale), con fusione RRF per il retrieval ibrido.
3. **Livello generazione**: Phi-3.5-mini-instruct riceve la query e i top-5 documenti come contesto e genera una risposta in linguaggio naturale con citazioni.

Funzione di retrieval ibrido con RRF

```
1 def retrieve_hybrid(query, faiss_index, bm25, documents, metadata, k=5):
2     # retrieval semantico (top-20) e lessicale (top-20)
3     sem = retrieve_semantic(query, faiss_index, documents, metadata, k=20)
4     bm = retrieve_bm25(query, bm25, documents, metadata, k=20)
5     # fusione RRF e restituzione dei top-k
6     return reciprocal_rank_fusion([sem, bm][:k])
7
8 def reciprocal_rank_fusion(results_list, k=60):
9     rrf_scores, doc_map = {}, {}
10    for results in results_list:
11        for item in results:
12            ix = item['idx']
13            rrf_scores[ix] = rrf_scores.get(ix, 0) + 1 / (k + item['rank'])
14            doc_map[ix] = item
15    sorted_docs = sorted(rrf_scores.items(),
16                        key=lambda x: x[1], reverse=True)
17    return [
18        {'rank': i+1, 'rrf_score': score, 'idx': ix,
19         'document': doc_map[ix]['document'],
20         'metadata': doc_map[ix]['metadata']}
21        for i, (ix, score) in enumerate(sorted_docs)
22    ]
```

Il retrieval ibrido parte da $k' = 20$ risultati per ciascun retriever prima della fusione, così da avere un pool di candidati sufficientemente ampio per il re-ranking.

4 Dataset e Configurazione

4.1 Dataset

Il dataset MaartenGr/arxiv_nlp contiene **44 949 paper scientifici NLP** da arXiv, con i campi title, abstract e year. Ogni documento è rappresentato come la concatenazione di titolo e abstract.

Caricamento e costruzione del corpus

```
1 dataset = load_dataset("MaartenGr/arxiv_nlp")
2 df_full = pd.DataFrame(dataset['train'])
3 # Documenti totali disponibili: 44,949
4
5 def build_corpus(df, n):
6     df_s = df.sample(n=n, random_state=42).reset_index(drop=True)
7     docs, meta = [], []
8     for _, row in df_s.iterrows():
9         title = str(row['title'])
10        abstract = str(row['abstract'])
11        docs.append(f"Title: {title}\n\nAbstract: {abstract}")
12        meta.append({'title': title, 'year': str(row['year'])})
13    return docs, meta
```

4.2 Configurazione dell’esperimento

L’esperimento varia la dimensione del corpus su cinque livelli:

$$N_{\text{DOCS}} \in \{1\,000, 5\,000, 15\,000, 25\,000, 44\,949\}$$

Per ogni configurazione si ricostruisce il corpus (campionamento casuale con seed fisso), si riaddestrano gli indici FAISS e BM25, e si rieseguo le valutazioni. Il gold standard è definito da cinque query NLP con parole chiave rilevanti:

Tabella 1: Query di valutazione e parole chiave del gold standard

Query	Parole chiave rilevanti
What is BERT and how does self-attention work?	bert, attention, transformer, language model, pre-training
How does word2vec represent word meaning?	word2vec, skip-gram, word embedding, word representation
What are the main challenges in machine translation?	machine translation, neural machine translation, nmt
How does topic modeling work?	topic model, lda, bertopic, topic, clustering
What is named entity recognition?	named entity, ner, entity recognition, sequence labeling

4.3 Modelli utilizzati

Tabella 2: Modelli e librerie del sistema

Componente	Modello/Libreria	Ruolo
Embedding	SciBERT (<code>allenai/scibert_scivocab_uncased</code>)	Retrieval semantico
Indice vettoriale	FAISS <code>IndexFlatIP</code>	ANN search
Retrieval lessicale	BM25Okapi (<code>rank_bm25</code>)	Keyword search
Fusione	Reciprocal Rank Fusion ($k = 60$)	Re-ranking ibrido
Generazione	Phi-3.5-mini-instruct (3.8B)	Produzione risposta

5 Implementazione del Retrieval

5.1 Retrieval semantico

Costruzione indice FAISS e retrieval semantico

```
1 def build_faiss_index(documents):
2     embeddings = embedding_model.encode(
3         documents, show_progress_bar=True,
4         batch_size=64, convert_to_numpy=True,
5         normalize_embeddings=True           # norma L2 = 1 (inner product =
6         coseno)
7     )
8     idx = faiss.IndexFlatIP(embeddings.shape[1]) # dimensione 768
9     idx.add(embeddings.astype(np.float32))
10    return idx
11
12 def retrieve_semantic(query, faiss_index, documents, metadata, k=10):
13     qe = embedding_model.encode(
14         [query], convert_to_numpy=True, normalize_embeddings=True
15     ).astype(np.float32)
16     sims, idxs = faiss_index.search(qe, k)
17     return [
18         {'rank': i+1, 'score': float(s), 'idx': int(ix),
19          'document': documents[ix], 'metadata': metadata[ix]}
20         for i, (s, ix) in enumerate(zip(sims[0], idxs[0]))
21     ]
```

Il retrieval semantico codifica la query con SciBERT e recupera i k documenti con massima similarità coseno. FAISS `IndexFlatIP` calcola l'inner product su tutti i vettori: essendo normalizzati, coincide esattamente con la similarità coseno.

5.2 Retrieval BM25

Costruzione indice BM25 e retrieval lessicale

```
1 def build_bm25_index(documents):
2     tokenized = [doc.lower().split() for doc in documents]
3     return BM25Okapi(tokenized), tokenized
4
5 def retrieve_bm25(query, bm25, documents, metadata, k=10):
6     scores = bm25.get_scores(query.lower().split())
7     top    = np.argsort(scores)[::-1][:k]
8     return [
9         {'rank': i+1, 'score': float(scores[ix]), 'idx': int(ix),
10          'document': documents[ix], 'metadata': metadata[ix]}
11         for i, ix in enumerate(top)
12     ]
```

BM25 tokenizza i documenti con semplice split su spazio e punteggia ogni documento in base alla frequenza normalizzata dei termini della query. Non richiede inferenza neurale, quindi è ordini di grandezza più veloce del retrieval semantico a runtime.

6 Generazione della Risposta

Prompt e generazione con Phi-3.5-mini-instruct

```
1 def rag_answer(query, results):
2     context_parts = [
3         f"[Paper {r['rank']}] {r['metadata']['title']}\n{r['document']}"
4         for r in results
5     ]
6     context = "\n\n".join(context_parts)
7     if len(context) > 3000:
8         context = context[:3000] + "\n[...truncated...]"
9
10    messages = [
11        {"role": "system", "content": (
12            "You are a helpful assistant specialized in NLP research. "
13            "Answer questions based only on the provided research papers. "
14            "Always cite the paper numbers [Paper X] you used. "
15            "If the context is insufficient, say so clearly."
16        )},
17        {"role": "user", "content": (
18            f"Context from research papers:\n\n{context}\n\n"
19            f"Question: {query}\n\n"
20            f"Answer based on the context above, citing paper numbers:"
21        )}
22    ]
23    prompt = tokenizer.apply_chat_template(
24        messages, tokenize=False, add_generation_prompt=True
25    )
26    output = llm(prompt)
27    full_text = output[0]['generated_text']
28    return full_text[len(prompt):].strip()
```

Il prompt è strutturato con ruolo `system` (vincola l'LLM all'uso del solo contesto fornito e richiede citazioni esplicite) e ruolo `user` (context + query). Il contesto è troncato a 3 000 caratteri per gestire la finestra di contesto del modello su CPU. La temperatura è $T = 0.3$ per favorire risposte deterministiche.

7 Metriche di Valutazione: Implementazione

Context Relevance e Answer Relevance

```
1 def context_relevance(query, results):
2     qe = embedding_model.encode(
3         [query], convert_to_numpy=True, normalize_embeddings=True)
4     snip = [r['document'][:200] for r in results]
5     de = embedding_model.encode(
6         snip, convert_to_numpy=True, normalize_embeddings=True)
7     return float(np.mean(de @ qe.T)) # media dei coseni query-snippet
8
9 def answer_relevance(query, answer):
10    embs = embedding_model.encode(
11        [query, answer], convert_to_numpy=True, normalize_embeddings=True)
12    return float(embs[0] @ embs[1]) # coseno query-risposta
```

```

1 def faithfulness_proxy(answer, results):
2     context_text = ' '.join(r['document'][:500] for r in results).lower()
3     words = re.findall(r'\b[a-z]{4,}\b', answer.lower())
4     content_words = [w for w in words if w not in sw] # sw = stopwords NLTK
5     if not content_words:
6         return 0.0
7     return sum(1 for w in content_words if w in context_text) / len(
        content_words)
8
9 def robustness_test(query, paraphrases, faiss_index, bm25,
10                     documents, metadata, k=5):
11     base_titles = {r['metadata']['title']}
12     for r in retrieve_hybrid(
13         query, faiss_index, bm25, documents, metadata, k):
14         overlaps = []
15     for para in paraphrases:
16         para_titles = {r['metadata']['title']}
17         for r in retrieve_hybrid(
18             para, faiss_index, bm25, documents, metadata, k):
19             overlaps.append(len(base_titles & para_titles) / k)
20     return float(np.mean(overlaps))

```

La **faithfulness proxy** è una stima lessicale della fedeltà: controlla che le parole contenutistiche della risposta siano ancorate al testo dei documenti recuperati. Non misura la correttezza logica (richiederebbe un NLI model), ma individua risposte “fuori contesto”.

La **robustezza** testa tre query con due parafrasature ciascuna, misurando l’overlap@5 tra i risultati della query originale e quelli delle varianti. Un overlap basso segnala instabilità del retriever a piccole variazioni lessicali.

8 Esperimento: Variazione di N_DOCS

8.1 Risultati per singolo N_DOCS

Le tabelle seguenti riportano i risultati completi per ciascuna configurazione.

Tabella 3: Risultati N_DOCS = 1000

Metrica	Base	Ibrido	Delta
Precision@5	0.160	0.400	+0.240
Context Relevance	0.716	0.718	+0.001
Answer Relevance	0.697	0.684	−0.014
Faithfulness	0.516	0.588	+0.072
Robustezza	—	0.233	—

Tabella 4: Risultati N_DOCS = 5000

Metrica	Base	Ibrido	Delta
Precision@5	0.240	0.560	+0.320
Context Relevance	0.724	0.714	−0.010
Answer Relevance	0.687	0.674	−0.014
Faithfulness	0.572	0.667	+0.095
Robustezza	—	0.100	—

Tabella 5: Risultati N_DOCS = 15000

Metrica	Base	Ibrido	Delta
Precision@5	0.160	0.520	+0.360
Context Relevance	0.713	0.719	+0.007
Answer Relevance	0.686	0.667	−0.018
Faithfulness	0.562	0.647	+0.084
Robustezza	—	0.033	—

Tabella 6: Risultati N_DOCS = 25000

Metrica	Base	Ibrido	Delta
Precision@5	0.200	0.440	+0.240
Context Relevance	0.720	0.730	+0.010
Answer Relevance	0.679	0.682	+0.003
Faithfulness	0.560	0.587	+0.027
Robustezza	—	0.000	—

Tabella 7: Risultati N_DOCS = 44949 (corpus completo)

Metrica	Base	Ibrido	Delta
Precision@5	0.120	0.360	+0.240
Context Relevance	0.719	0.726	+0.007
Answer Relevance	0.678	0.666	−0.012
Faithfulness	0.496	0.569	+0.074
Robustezza	—	0.033	—

8.2 Riepilogo comparativo: Precision@5

Tabella 8: Precision@5 al variare di N_DOCS

N_DOCS	Base	Ibrido	Delta	Miglioramento
1 000	0.160	0.400	+0.240	+150.0%
5 000	0.240	0.560	+0.320	+133.3%
15 000	0.160	0.520	+0.360	+225.0%
25 000	0.200	0.440	+0.240	+120.0%
44 949	0.120	0.360	+0.240	+200.0%

8.3 Dettaglio per query: Precision@5 ibrido

Tabella 9: P@5 ibrido per singola query al variare di N_DOCS

Query	1K	5K	15K	25K	44.9K
BERT / self-attention	0.20	0.40	0.40	0.00	0.20
word2vec	0.40	0.60	0.60	0.60	0.40
machine translation	0.20	0.40	0.60	0.60	0.40
topic modeling	0.40	0.60	0.20	0.20	0.20
named entity recog.	0.80	0.80	0.80	0.80	0.60
Media	0.400	0.560	0.520	0.440	0.360

8.4 Robustezza al variare di N_DOCS

Tabella 10: Robustezza (overlap@5) al variare di N_DOCS

N_DOCS	Robustezza
1 000	0.233
5 000	0.100
15 000	0.033
25 000	0.000
44 949	0.033

9 Risultati e Discussione

9.1 Il retrieval ibrido migliora sistematicamente la Precision@5

Il dato più netto è che il sistema ibrido supera il baseline semantico in *tutte e cinque* le configurazioni di N_DOCS, con miglioramenti che vanno da +120% a +225%. L'ibrido porta la Precision@5 media da valori tra 0.120 e 0.240 a valori tra 0.360 e 0.560.

Questo conferma l'ipotesi di partenza: BM25, catturando i termini tecnici esatti (es. "NER", "LDA", "skip-gram"), recupera documenti che il solo retrieval semantico manca, perché SciBERT tende a generalizzare i termini tecnici verso concetti più astratti. La fusione RRF bilancia i punti di forza dei due retriever.

9.2 Andamento non monotono della Precision@5 con N_DOCS

La Precision@5 ibrida non cresce monotonamente all’aumentare del corpus:

- Picco a N=5 000 (P@5=0.560): il corpus è abbastanza grande da contenere paper rilevanti, ma abbastanza piccolo da non essere “inquinato” da troppi documenti distanti.
- Calo a N=44 949 (P@5=0.360): l’aumento esponenziale del numero di documenti incrementa il “rumore” nel retrieval — BM25 soffre di più con vocabolari molto grandi, e anche FAISS deve discriminare tra molti più candidati simili.

Questo suggerisce che per corpus di paper NLP molto grandi, sarebbe necessario pre-filtrare per sotto-dominio (es. clustering gerarchico) prima del retrieval.

9.3 Context Relevance e Answer Relevance: differenze minime

La Context Relevance (CR) e l’Answer Relevance (AR) mostrano differenze piccole tra base e ibrido (tipicamente $|\Delta| < 0.02$). Questo è atteso: entrambe le metriche sono basate sugli stessi embeddings SciBERT usati anche per il retrieval semantico, creando una parziale circolarità. Il retrieval semantico massimizza già la similarità coseno con la query, quindi CR e AR non riescono a discriminare ulteriormente tra i due sistemi.

L’AR ibrida è spesso leggermente inferiore a quella base: i documenti BM25 sono meno “semanticamente centrati” sulla query (recuperano per parole chiave, non per embedding), quindi il contesto risultante, pur più preciso nel contenuto fattuale, può essere meno coeso, portando l’LLM a risposte con embedding più dispersi.

9.4 Faithfulness: il sistema ibrido produce risposte più fedeli

La Faithfulness proxy è sistematicamente più alta per il sistema ibrido in tutti gli esperimenti (+0.027 a +0.095). Questo è il risultato più interpretabile: i documenti recuperati dall’ibrido sono più aderenti al dominio tecnico specifico della query, quindi l’LLM usa più termini che compaiono letteralmente nel contesto, riducendo le allucinazioni lessicali.

9.5 Robustezza: cala drasticamente con N_DOCS

La robustezza è alta a N=1 000 (0.233) e crolla a N=25 000 (0.000). Questo è contro-intuitivo a prima vista: con più documenti ci si aspetterebbe retrieval più stabile. La spiegazione è che con corpus piccoli i risultati sono dominati da pochi paper molto rilevanti, stabili anche a variazioni della query. Con corpus grandi, la minima variazione lessicale della query sposta i confini di un denso spazio di candidati, portando a ranking molto diversi.

La robustezza bassa al corpus grande segnala la necessità di tecniche di *query expansion* o ensemble di retriever più sofisticati per sistemi di produzione su grandi knowledge base.

9.6 Query con performance diverse

La query “named entity recognition” è quella con P@5 più alta e stabile (0.60–0.80): il termine “NER” è molto specifico e BM25 lo gestisce perfettamente. La query “BERT /

self-attention” ha P@5 bassa a $N=25\,000$ (0.00): a quella dimensione del corpus, il retrieval trova paper che parlano di attention in generale, senza includere paper specificatamente su BERT tra i top-5.

9.7 Nota metodologica: revisione dei corpus campionati e delle metriche

I numeri e le interpretazioni discussi in questa sezione sono stati prodotti da una prima versione degli script che presentava alcuni limiti metodologici, corretti in una revisione successiva del codice (`lab5rag.py` e `lab5_hybrid.py`):

- **Campionamento non annidato:** ogni corpus a N_DOCS diverso era ottenuto con una chiamata indipendente a `df.sample(n=N, random_state=42)`. Poiché pandas non garantisce che un campione più piccolo sia un sottoinsieme di uno più grande, un paper poteva essere presente nel corpus da 5000 documenti e assente in quello da 44949 (o viceversa) per puro effetto del campionamento, non per un reale “degrado del retrieval al crescere del corpus”. Questo rimette in discussione in particolare le spiegazioni sopra proposte per il picco a $N=5\,000$, il calo a $N=44\,949$ e il crollo della robustezza a $N=25\,000$: non si può escludere che riflettano semplicemente quali paper sono finiti in quali campioni, più che un effetto genuino della dimensione del corpus. Nella revisione, il dataset viene mischiato una sola volta e ogni corpus prende i primi N documenti di quell’ordine fisso, così che i corpus siano sovrainsiemi annidati gli uni degli altri e l’effetto di N_DOCS sia isolato correttamente.
- **Precision@k non uniforme fra i due script:** lo script base cercava le keyword in titolo+abstract, quello ibrido solo nel titolo (una misura più severa): il confronto diretto dei numeri fra i due script, per come erano definiti, non era pienamente equo. Nella revisione entrambi gli script cercano le keyword in titolo+abstract.
- **Matching a substring:** keyword come “ner” o “bert” potevano matchare falsi positivi (“corner”, “RoBERTa”), gonfiando la Precision@k. Nella revisione il matching è per parola (o frase) intera.
- **Tokenizzazione BM25 naive** (`split()` su spazi, senza rimuovere la punteggiatura attaccata ai token): può aver penalizzato BM25 nei confronti con l’embedding semantico. Nella revisione la tokenizzazione usa `\w+`.
- **Troncamento del contesto sull’intero blocco** (`MAX_CONTEXT=3000` caratteri, circa 2–3 abstract): con k grande, l’LLM vedeva comunque solo i primi 2–3 paper, rendendo il confronto “al variare di k ” meno informativo di quanto sembrasse; inoltre la faithfulness proxy leggeva una finestra di contesto (500 caratteri) diversa da quella realmente vista dall’LLM (3000 caratteri), sottostimandola per costruzione. Nella revisione il troncamento è per singolo paper, e la faithfulness usa la stessa finestra del prompt.
- **Generazione non deterministica nonostante il commento in codice:** gli script usavano `do_sample=True` con `temperature=0.3`, quindi le risposte (e le metriche Answer Relevance/Faithfulness da esse derivate) potevano variare fra un’esecuzione e l’altra, contraddicendo il commento che le descriveva come deterministiche. Nella revisione la generazione è greedy (`do_sample=False`).

Il fix più rilevante è il primo: rende finalmente interpretabile la curva “Precision@k su BERT al variare di N_DOCS” discussa in `lab5rag.py`, isolando l’effetto della dimensione del corpus da quello, puramente casuale, della presenza/assenza di un singolo paper nel campione. Le tabelle e le conclusioni quantitative di questo capitolo andrebbero quindi rigenerate rilanciando gli script aggiornati prima di essere usate come risultato finale; le osservazioni qualitative (il retrieval ibrido recupera meglio i termini tecnici esatti, la Faithfulness beneficia della fusione, la Context Relevance è parzialmente circolare rispetto al retrieval semantico) restano valide indipendentemente dal fix.

10 Conclusioni

Il laboratorio ha implementato un sistema RAG ibrido su un corpus di paper scientifici NLP, dimostrando che la combinazione di retrieval semantico (SciBERT + FAISS) e lessicale (BM25) tramite Reciprocal Rank Fusion porta a miglioramenti consistenti rispetto al solo retrieval semantico. I numeri esatti riportati sotto sono quelli della prima esecuzione (Sezione 9.7): la direzione qualitativa dei risultati è considerata solida, ma andrebbe confermata rilanciando l’esperimento con gli script corretti prima di essere citata come risultato definitivo.

I risultati principali sono:

1. **Il retrieval ibrido migliora la Precision@5 in modo sistematico:** il delta medio su tutte le configurazioni è +0.280, con un miglioramento percentuale che va da +120% a +225%.
2. **La dimensione ottimale del corpus è N=5 000:** il picco di P@5 ibrido (0.560) si raggiunge a 5 000 documenti, suggerendo che per questo tipo di query un corpus più focalizzato è preferibile a un corpus completo.
3. **La Faithfulness beneficia del retrieval ibrido:** l’ibrido produce risposte più ancorate al testo dei documenti recuperati, riducendo le allucinazioni lessicali.
4. **La robustezza è inversamente correlata alla dimensione del corpus:** questo apre la strada a lavori futuri su query expansion, diversificazione dei risultati e indici gerarchici per grandi knowledge base.

I limiti principali riguardano il gold standard (definito manualmente e limitato a 5 query), il costo computazionale su CPU (necessario in assenza di GPU), e la faithfulness proxy che è una stima lessicale e non valuta la correttezza logica delle risposte.